



Design the network. Compile the circuit. See what it costs.

— THE GAP

Circuit frameworks, distributed compilers and network execution environments each solve one layer well. Nothing connects them, so the question that matters most — how the optical network changes what the application achieves — falls between them.

- **MQT Bench** — benchmark circuits, by name and qubit count
- **pytket-dqc** — compiles and distributes over the network
- **Qoala** — node scheduling, TU Delft · arXiv:2502.17296
- **NetSquid** — discrete-event execution of the network

— QNEST

The connective tissue.

Rather than reimplementing any layer, QNEST wires established tools into one workflow and adds what was missing: optical network design, all-photonic and memory-assisted switching protocols, and a graphical environment over the whole run.

— RUNS ON

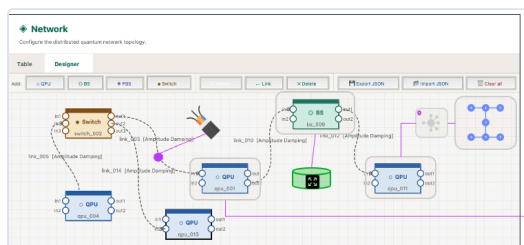
Windows · macOS · Linux. Standalone application, no command line required.

QNEST is an open-source graphical toolkit for optical quantum data centers. Build a topology out of QPUs, switches and fiber, distribute a quantum circuit across it, schedule the remote gates, and run the whole thing on established tools — MQT Bench, pytket-dqc, Qoala and NetSquid — wired into one workflow, with the optical switching protocols and the interface on top.

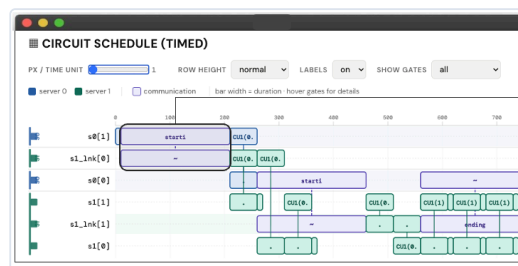
— THE WORKFLOW

Five stages, in order, end to end.

- 01 Circuit · MQT Bench**
Choose the benchmark circuit and the number of qubits for the run, and set the duration of every gate and process.
- 02 Network**
Define the topology and the connectivity between QPUs: beam splitters, switches, quantum memories and optical links, with qubit counts, T1 and T2, gate and readout errors, or calibration data from a real backend.
- 03 Distribute · pytket-dqc**
pytket-dqc compiles the circuit against that network and distributes it over the QPUs, exposing remote-gate dependencies and entanglement demand.
- 04 Schedule · Qoala + NetSquid**
The distributed circuit is passed to the QPUs and scheduled by strategies defined in Qoala and NetSquid, then drawn as an interactive Gantt chart exported as standalone HTML.
- 05 Run · switching protocols**
Our switching protocols are applied for either an all-photonic or a memory-assisted architecture. Results come back as detailed tables, down to the individual event.



NETWORK DESIGNER



CIRCUIT SCHEDULE

PARAMETERS

— WHAT A RUN TELLS YOU · REPORTED AS DETAILED TABLES

Entanglement fidelity

Quality of the pairs delivered to remote gates.

Job execution time

Wall-clock length of the job, network included.

Communication overhead

Time spent moving information, not computing.

Resource utilization

How hard generation and memory are worked.

Penalty delay

Waiting on an unavailable entangled pair.